Final Report

[redacted]

5/10/2026

Introduction

This capstone project focuses on the development of enemy artificial intelligence (AI) systems used in modern video game development. Enemy AI refers to the techniques and systems implemented to simulate intelligent behavior in non-player characters (NPCs) across interactive gaming environments. One common approach involves the use of finite state machines (FSMs), which define enemy behaviors within specific states and govern the transitions between those states based on changing in-game conditions. Building upon the finite state machines implemented during the previous semester, this project expands the enemy AI system through the integration of behavior trees, allowing for more dynamic and scalable decision-making processes. Behavior trees organize enemy logic into a hierarchical structure of nodes, allowing enemies to evaluate conditions, select actions, and execute behaviors in a controlled sequence. Compared to FSMs, behavior trees can provide a more modular and scalable approach for creating complex enemy behavior.

For this project, enemy AI was developed for a single-player, top-down 2D game in the Godot engine, where the player traverses a large environment populated by hostile entities. These enemies utilize finite state machines and behavior trees to determine their actions in response to gameplay variables, while a pathfinding algorithm enables efficient navigation throughout the game world and movement toward designated targets.

Project Timeline

In the previous semester, our goals for the project primarily revolved around creating enemy AI that used FSMs as a model to facilitate decision-making. This was accomplished alongside the development of enemy pathfinding through the use of the A* algorithm and the

creation of a tool to manually change the state enemies were in. Initially, we anticipated these tasks would take far less time than they ended up taking, resulting in many goals being pushed back until later in the semester or not being accomplished at all.

During this semester, our primary goal was to expand upon the systems used by enemy AI, further visualize enemy behavior, and add more gameplay variety. Since the earlier version of the project relied heavily on FSMs, one of our main goals was to create the scaffolding for behavior trees that could support NPCs with a wider range of actions. Alongside this, both the FSMs and behavior trees used by the enemy AI needed to be properly showcased through the use of a visualization tool, making it easier to debug and demonstrate enemy behavior. Additional goals included designing new enemy types that use behavior trees, creating new player classes, integrating a Navigation Mesh, and expanding upon the map created in the previous semester. Together, these goals were intended to improve both the technical depth of the project and the gameplay experience provided to the player. The timeline for this semester is depicted in Table 1, located in Appendix A, showing how these goals were planned and completed throughout the semester.

Project Goals

Behavior Tree

In the previous semester, our team implemented FSMs to provide structure for the enemy logic used by the NPCs. While FSMs were useful for organizing enemy behavior into defined states, this semester our team aimed to expand upon our enemy AI by building the scaffolding for behavior trees. The article *NPC Control Across Algorithm Classes: Static Logic, Evolutionary, Reinforcement Learning, and Hybrid Generative Methods*, by Krieolin Naidoo and Duncan Coulter, discusses several approaches to NPC control, including behavior trees. The

authors describe behavior trees as a modular approach to NPC decision-making that organizes logic into a tree-like structure made up of different node types, such as selector, sequence, action, condition, and decorator nodes. Naidoo and Coulter also explain that behavior trees are commonly used in game development because they allow designers to create a variety of behaviors for NPCs while maintaining predictable control over NPC decision-making (Naidoo and Coulter).

The switch to behavior trees was made due to the scalability issues that arose from the FSMs created for enemies in the previous semester. While FSMs are effective for organizing enemies with a small number of states, they become more difficult to manage as the number of states increases. Adding a wider range of enemy actions would require additional states and transitions, which would quickly make the FSM difficult to maintain. Behavior trees address this issue by organizing enemy logic into a structure of reusable nodes rather than relying on direct transitions between every possible state. This allows each enemy to have its own unique set of behaviors while still using the same underlying node system. Unlike FSMs, which move between predefined states, behavior trees evaluate conditions and actions in a structured order, allowing enemies to make decisions in a more modular and scalable way.

To accomplish this, our team planned to create the scaffolding for behavior trees by designing a system of reusable node classes. These included composite nodes, such as selectors and sequences, as well as leaf nodes, which include action and condition nodes. Selector nodes would allow an NPC to choose between multiple possible nodes, while sequence nodes would require a set of nodes to succeed in order. Action nodes would perform specific enemy behaviors, such as moving or attacking, while condition nodes would check game state information, such as whether the player is within a set range of the NPC. Decorator nodes were also implemented to modify or restrict how certain behaviors were executed. This structure was intended to make it easier to create new NPCs with a larger variety of actions they can perform.

The scaffolding for the behavior tree was implemented over the course of multiple sprints, which involved planning each node type and testing node behavior. Before any NPCs could use a behavior tree, our team first needed to determine how navigation from node to node would work throughout the tree. This was accomplished through the use of success, failure, and running states, which allowed nodes to communicate their results to one another. Once the basic structure was created, additional time was spent testing how action and condition nodes would utilize existing logic for enemy attacking, pathfinding, and targeting.

Enemy Types

During the development of the behavior tree scaffolding, the first new enemy type was created using the FSM system developed for enemies in the previous semester. This enemy was designed as a Slime enemy, with its main behavior focused on spawning smaller versions of itself after reaching zero health. The Slime enemy only took one sprint to implement because it used the previously existing FSM structure rather than the new behavior tree system. However, due to the limited information available on implementing this enemy type in Godot, AI tools were used to research possible approaches and help guide the design of the splitting mechanic.

The second enemy to be added was the Colossus enemy, which was designed to use the newly added behavior tree system. This enemy was originally planned to focus on resurrecting nearby dead enemies. However, after issues with art assets and team discussions about plans for a Pyromancer player class, the goal of this enemy shifted toward spawning area-of-effect (AOE) hazards that would force the player to constantly move out of attack areas. The Colossus enemy still had a resurrection attack developed for it, but the majority of its attacks focused on spawning AOE hazards rather than interacting with dead enemies on the map. To add more variety to the boss's attacks, randomized selector nodes and cooldown decorator nodes were implemented. The randomized selector nodes were created to select a

random attack sequence for the Colossus to prioritize when targeting the player, while the cooldown decorator nodes wrapped around child nodes and returned failure if those nodes were still on cooldown. Parallel nodes were also implemented to allow action nodes for attacking and playing animations to run at the same time. This helped make the enemy's behavior tree more robust and allowed its attacks to feel more polished during gameplay.

The third enemy to be added was the Assassin enemy, which was designed to use a behavior tree and teleport action nodes to pressure the player. Initially, the Assassin was implemented to teleport only once the player fell below a certain health threshold. However, after testing, this behavior made the enemy feel less active during combat. To make the Assassin feel more reactive, the teleport behavior was changed to use a cooldown decorator node, allowing the enemy to teleport every few seconds instead. The Assassin was also designed to have a continuous attack animation if the player remained within attack range. This required the implementation of a loop decorator node, which allowed the behavior tree to repeatedly execute the child node it was wrapping, either a set number of times or until the child node failed. This helped make the Assassin feel more aggressive while also expanding the functionality of the behavior tree system.

Both the Colossus and Assassin enemies took multiple sprints to properly develop, and both required more time than originally anticipated. Much of this extra time came from designing their unique behaviors, adjusting how their actions interacted with the player, and ensuring that their behavior trees worked correctly with existing enemies. However, adding more logic to each enemy's behavior tree was not a major challenge due to the modular nature of behavior trees. Since each behavior could be separated into individual nodes or smaller subtrees, adding new enemy actions to an enemy's behavior tree was often a simple process. This made it easier to continue building on each enemy's logic without having to redesign the entire system.

The final enemy type planned for development was a knockback enemy. This enemy was originally intended to be its own separate enemy type with a focus on pushing the player

away. However, due to time constraints, the knockback enemy was never fully created as an individual enemy. Instead, the knockback functionality was added to the current roster of enemies, allowing existing enemies to push the player back while dealing damage to them. This required slight changes to player movement so the player could be knocked backward without breaking normal movement controls. While the original enemy was not completed, the added knockback functionality still helped increase gameplay variety by making enemy attacks feel more impactful.

Enemy AI Visualization

The enemy AI visualization tool was created to address one of the project's major goals: making enemy decision-making easier to observe, debug, and explain. Since the project used both finite state machines and behavior trees, the team needed a way to visually represent how enemies moved between states or nodes during gameplay. This goal was supported by the ideas discussed in *NCAIt: Alternatives and Difference Visualizations for Behavior Trees in Game Development Learning* by Yousuf Hossain and Loutfouz Zaman. The article explains that behavior trees are commonly used as visual modeling tools for game AI and that real-time visualization can help developers compare, test, and understand different NPC behaviors more effectively (Hossain and Zaman). Our team wanted the visualization tool to serve as both a debugging system and a demonstration tool. During development, it would allow team members to see what state or behavior an enemy was currently using, making it easier to identify issues with enemy logic. For example, if an enemy failed to attack, chase, or transition correctly, the visualizer could help determine whether the issue came from the AI logic itself or from another system. From a presentation standpoint, the tool also allowed the team to show how enemy AI functioned beneath the surface, giving viewers a clearer understanding of how FSMs and behavior trees controlled NPC behavior.

The original plan for the visualization tool was to create a GUI that could display an enemy's active AI structure while the game was running. For enemies using FSMs, this meant showing the current state and any transitions occurring between states. For enemies using behavior trees, the goal was to display the tree structure and highlight which nodes were currently being evaluated or executed. Similar to the visualization approach discussed by Hossain and Zaman in the NCAIt article, the tool was intended to make AI behavior easier to understand by turning abstract logic into a visual format (Hossain and Zaman).

Several challenges arose during the development of the visualization tool. The first major challenge was presenting AI information in a way that was useful without overwhelming the screen. Versions of the debug interface developed in the previous semester relied solely on displaying which state the enemy was in and not the entire FSM, which made it difficult to quickly understand enemy behavior during active testing. As a result, the visualization system had to be redesigned to improve clarity, organization, and usability. Another challenge was keeping the visualizer updated in real time while the game continued running. Since enemies could change states or behavior tree nodes quickly during combat, the interface needed to display this information dynamically without interfering with gameplay. The team also had to account for the differences between FSMs and behavior trees, since each system required a different visual structure.

The final version of the visualization tool represented a major improvement over the original debugging tool. The updated interface provided a clearer way to monitor enemy behavior, observe state changes, and track behavior tree execution during gameplay. The system also helped improve the development workflow by making enemy AI issues easier to locate. While the tool required redesigns and testing over multiple sprints to become usable, it ultimately supported one of the project's larger goals: displaying enemy AI in a way that made enemy systems easier to debug and understand.

Navigation Mesh

Last semester, the project's pathfinding system was fully manually implemented with the goal of creating unique interactions between the environment, the player, and enemy AI behavior. Developing a custom solution allowed for greater control over enemy navigation logic and provided the team with an opportunity to explore the underlying mechanics of pathfinding systems in game development. The original implementation was designed to integrate closely with the enemy finite state machines (FSMs) and support more specialized movement behaviors throughout the game world.

As development progressed, however, the manual pathfinding implementation proved significantly more time-consuming than anticipated and continued to present unresolved issues related to path consistency, obstacle handling, and overall reliability. Due to the team's limited prior experience with advanced pathfinding algorithms, Godot Engine, and game AI systems, the decision was made to migrate the project to Godot Engine's built-in Navigation Mesh (NavMesh) system. Transitioning to the integrated solution allowed development efforts to be redirected toward higher-priority gameplay systems while improving project stability and maintainability. Additionally, the modular structure of the original implementation simplified the migration process by allowing several previously developed methods to be reused within the new navigation framework, reducing redevelopment time and easing integration with the existing AI systems.

Enemy pathfinding serves a critical role within the project, as enemy movement directly impacts gameplay responsiveness, combat interactions, and overall player immersion. In the article *A*-based Pathfinding in Modern Computer Games*, Xiao Cui and Hao Shi identify pathfinding as one of the most significant and challenging problems in game artificial intelligence, particularly as game environments increase in complexity. Reliable navigation behavior is essential to ensuring enemies can effectively pursue the player, maneuver around obstacles, and respond dynamically to changing in-game situations. Without a dependable

pathfinding system, enemy AI behavior can appear inconsistent or unrealistic, negatively affecting gameplay quality and player experience (Cui and Shi).

The NavMesh system was implemented by utilizing Godot Engine's built-in Navigation Mesh framework, which defines traversable areas of the game world through a 2D polygonal navigation map. Once these walkable surfaces are established, enemies are able to calculate efficient paths toward a target while automatically avoiding walls and other environmental obstacles. Internally, Godot utilizes the A* pathfinding algorithm to determine optimal movement routes, enabling enemy AI to navigate the environment in a reliable and context-aware manner. This implementation aligns with existing research describing A* as one of the most widely used and effective pathfinding algorithms in modern game development due to its ability to efficiently calculate optimal routes while navigating around obstacles. By utilizing the NavMesh system, the project was able to achieve consistent, wall-aware enemy navigation without requiring continued development of a fully custom pathfinding solution.

Player Classes

Players currently have access to multiple playable classes, each designed to support a distinct combat style and player experience. Alongside the original melee-focused Samurai, both the Ranger and Pyromancer classes were added to increase gameplay variety and give players different approaches to combat. The Samurai focuses on aggressive close-range attacks, the Ranger emphasizes maintaining distance from enemies, and the Pyromancer uses fire-based abilities to create AOE fields that burn enemies. Each class includes light, heavy, and skill attacks, alongside stamina management, damage visual effects, and animation sequences.

To support the addition of new classes, the player architecture was refactored into an inheritance-based class system. Previously, most functionality was contained within a single large script designed specifically for the Samurai, which made it difficult to expand the player roster. The updated structure uses a shared "PlayerBase" script to handle common systems

such as movement, attacking, health, stamina, and animations. Individual classes such as the Samurai, Ranger, and Pyromancer inherit from this parent class while maintaining their own scripts for class-specific combat behavior and abilities. This modular approach made the player system easier to expand, maintain, and adapt for future classes.

Each class was also given unique mechanics to further distinguish its playstyle. The Samurai class now has a unique block skill that can mitigate damage taken. The Ranger uses quick arrow attacks, stronger charged shots, and a dodge to maintain distance from enemies. The Pyromancer relies on fire projectiles, explosive attacks, and lingering area effects to damage multiple enemies at once. In addition to their attacks, each class applies a unique status effect that reinforces its combat role. The Samurai applies Bleed to deal damage over time, the Ranger can Stun enemies to create space, and the Pyromancer inflicts Burn, which can stack and deal continuous damage. Together, these systems provide a wider range of combat experiences and encourage players to approach encounters differently depending on their chosen class.

Map Expansion

Last semester, the primary goal of the environment design was to establish the foundational structure of the game world by developing the initial half of the map. This portion of the map was carefully designed to define the core gameplay space, including enemy encounters, basic navigation flow, and the initial layout of major environmental landmarks. Particular attention was given to shaping the player's early experience, ensuring that movement through the world felt intuitive while still allowing for exploration and encounter variation within the established space.

This semester, development efforts focused on expanding the existing environment by completing the remaining portion of the map and significantly increasing the overall scale of the game world. Rather than redesigning the established structure, the new areas were built to

extend and complement the existing layout, effectively doubling the playable space while maintaining consistency in environmental style, encounter design, and navigation flow.

While the map was expanded this semester, environment design was not the primary focus of the project, as the main emphasis remained on enemy artificial intelligence systems and gameplay behavior. As a result, map development functioned as a supporting component rather than a central objective, intended primarily to provide a functional and engaging space in which AI systems could operate and be evaluated effectively.

To implement the expanded environment, the team continued using Godot Engine's built-in 2D tilemap tools in combination with Tiled Map Editor. This allowed for efficient construction of large environments while maintaining consistency in tile placement, collision boundaries, and navigation regions. Environmental features such as main pathways and points of interest were integrated directly into the tilemap structure, ensuring compatibility with the Navigation Mesh system and supporting reliable enemy movement and pathfinding behavior across the expanded game world.

Accomplishments

The complete MoSCoW analysis for this semester is depicted in Table 2, found in Appendix B. Throughout the semester, the team successfully completed the majority of the project's Must Have objectives. These accomplishments included the implementation of behavior trees for enemy AI, the development of an enemy state visualizer, and the integration of NavMesh for pathfinding. These systems represented core components of the project and significantly expanded both the functionality and scalability of the game's AI architecture.

In addition to the primary objectives, the team also completed several secondary goals that enhanced gameplay systems and improved overall project quality. These accomplishments included the implementation of status effects, the expansion of the game map, and the

development of a player class selection system. Collectively, these additions improved gameplay variety and added environmental depth to the project.

Despite the completion of most primary objectives, several planned features were not fully implemented during the semester. One major goal that was not completed involved the development of an enemy state capable of dynamically affecting pathfinding behavior or enemy decision-making. This feature was ultimately deprioritized because other core systems required more development time than originally anticipated.

Additionally, several lower-priority features remained unfinished, including a settings menu, player statistics, experience systems, sound effects, and player state visualization tools. These features were categorized as lower priority within the project scope and were deferred in favor of completing critical gameplay and AI systems. In the case of player state visualization, continued development was determined to be unnecessary after further evaluation of the system's usefulness within the project.

Future Work

If our team were given a third semester to continue working on this project, our primary goals would be to improve the overall player experience and incorporate more complex enemy AI within behavior trees. After playtesting, some of the main issues identified by users were player confusion about their location on the map and player sustainability throughout the game. These issues could be addressed by adding a mini-map feature to show the player's current position within the larger environment or by creating a new menu screen that displays the areas of the map the player has discovered. Player sustainability could also be expanded into a larger player progression model. This could include stat improvements upon killing a certain number of enemies, as well as upgrades to player abilities that are unlocked after defeating specific enemies, such as bosses.

Another major area of future work would involve expanding the complexity of enemy AI. This could be accomplished by incorporating a state machine into the player class to monitor the player's current actions. By tracking whether the player is attacking, using a skill, or attempting to run away, enemies could make more informed decisions through their behavior trees. This would allow enemies to respond more dynamically to the player's behavior and create more engaging combat encounters.

Conclusion

The purpose of this capstone was to create a game environment that incorporated complex enemy AI systems capable of making decisions and navigating effectively during gameplay. By implementing finite state machines, behavior trees, and incorporating a Navigation Mesh into enemy pathfinding, the project introduced enemies with unique behaviors and movement patterns that helped create a more engaging gameplay experience. The addition of multiple player classes also expanded gameplay variety by giving players different ways to approach combat and other in-game challenges. Another important aspect of the project was the enemy AI visualizer, which provided a clearer view of how enemy logic functioned within both finite state machines and behavior trees. Throughout development, the team faced several technical and design challenges, especially with pathfinding, AI scalability, and integrating new systems into the existing project. These challenges required ongoing testing, problem-solving, and adjustments to the project's priorities. Despite these difficulties, the team completed the majority of its primary objectives and created a foundation that could be expanded upon in future development. Overall, this capstone provided meaningful experience with enemy AI, game system design, collaborative development, and technical problem-solving in the context of modern game development.

Appendix A: Sprint Timeline

Sprint 1	Sprint 2
● Research into hierarchical state machines ● Research behavior tree design for boss AI ● Research Navigation Mesh ● Create a Navigation Mesh test environment ● Fix bugs from the previous semester ● Art asset collection	● Build scaffolding for behavior tree ● Integrate Navigation Mesh into the game ● Refactor player class and abilities ● Develop a ranged player class ● Implement Bleed and Stun status effects

Sprint 3	Sprint 4
● Complete Colossus boss's behavioral tree nodes ● Create the Slime enemy ● Begin development on Colossus boss ● Add bleed and stun status effects to enemy roster ● Begin behavior tree visualization system	● Poster preparations ● Finish development on Colossus boss ● Implement projectiles splitting on collision ● Expand the dungeon map to boss arena ● Add player ability to mitigate or avoid damage ● Create a Knockback enemy (time permitting) ● Continue work on visualizing enemy states

Sprint 5	Sprint 6
● Create an Assassin enemy ● Allow for the player to movement cancel abilities ● Allow for the player to regain health ● Add controller support to the game ● Continue work on first boss if needed	● Polish and populate the map ● Develop a Burning state that enemies can enter ● Add third "pyromancer" player class ● Continue work on player abilities if needed ● Continue work on basic enemies if needed

Sprint 7	Sprint 8
● Outline of what we are gonna talk about. ● Finish work on map ● Finish work on enemy state ● Finish work on behavior visualization ● Allow for fire to spread across map tiles ● Continue work on Burning state ● Continue work on third player class	● Presentation preparations ● Player and enemy stat balancing ● Bug fixing

Table 1: Sprint Timeline Appendix

Appendix B: MoSCoW Analysis

Must Have	Should Have	Could Have	Won't Have
Behavioral Trees	Navigation Mesh	Class Selection Menu	Save game progress
FSM	Enemy Inflicted States (Status Effects)	Fluid Animations	Save user credentials
State Visualizer	Environmental Interactions	Controller Support	Login menu
Pathfinding	Pyromancer Player Class	Player Experience	Prevent piracy
Player Inflicted States (Status Effects)	Expanded Map	Player Statistics	
State that Affects Pathfinding	Settings Menu	In-game Checkpoints	
New Enemies (FSM and BT)		Visualize Player States	
Ranged Player Class		Music and Sound Effects	

Table 2: MoSCoW Analysis

Works Cited

Cui, Xiao and Shi, Hao. *A*-based Pathfinding in Modern Computer Games*. https://www.researchgate.net/publication/267809499_A-based_Pathfinding_in_Modern_Computer_Games.

Hossain, Md. Yousuf, and Loutfouz Zaman. *NCAlt: Alternatives and Difference Visualizations for Behavior Trees in Game Development Learning.* Proceedings of the ACM on Human-Computer Interaction, vol. 6, no. CHI PLAY, 25 Oct. 2022, pp. 1–31, https://doi.org/10.1145/3549508.

Naidoo, Krieolin Navindran, and Duncan Anthony Coulter. *NPC Control across Algorithm Classes: Static Logic, Evolutionary, Reinforcement Learning, and Hybrid Generative Methods.* Proceedings of the 2025 8th International Conference on Computational Intelligence and Intelligent Systems, 21 Nov. 2025, pp. 58–69, https://doi.org/10.1145/3787256.3787265.